

Entity Relationship Diagram

Entities, relationships and the link between the catalog and the 3DCityDB city model.

Project	Digital Twin Catalog (NARAGA 3D Catalog), reusable 3DCityDB template
Document	Entity Relationship Diagram
Version	0.1 draft for review, 4 October 2026
Author	Fairuz Akmal Pradana
Repository	github.com/fakmalpradana/naraga-twin-db

Contents

Introduction

How to read the diagrams

Overview

Catalog core

Import traceability

Link to the city model

Why a tag and not a link table

3DCityDB v5 facts the model relies on

Audit

Code lists

Read model

Introduction

This document shows the entities of the Digital Twin Catalog and how they relate to the 3DCityDB city model. Diagrams are split by topic so each one stays readable. The table and column list of every entity is in the *Database Schema* document, which is generated from the live database.

Verified against 3DCityDB v5.1.4 (image `3dcitydb/3dcitydb-pg:16-3.4-5.1.4`) on 3 October 2026.

How to read the diagrams

Notation	Meaning
<code>\ \ --o{</code>	One parent row has zero or many child rows; the child must have a parent
<code>\ o--o{</code>	Optional parent: the foreign key may be empty
PK, FK, UK	Primary key, foreign key, unique key
Dotted line	Logical link through a tag, not a database foreign key
<code>citydb_</code> prefix	Table of the 3DCityDB schema <code>citydb</code> (owned by 3DCityDB, shown for context)

To keep the diagrams readable, two groups of columns are left out:

- **Audit columns** on `dataset`, `import_job`, `layer` and `tileset`: `created_at`, `created_by`, `updated_at`, `updated_by`.
- **Lookup references** to code lists. They are listed in the section *Code lists*.

Entity Relationship Diagram Page 4 of 12

Context	Entities
Administration and Audit	audit.change_log , Django tables in schema app

Catalog core

A dataset holds many layers (one per theme and LOD). A layer has many tileset builds, and at most one is active.

Figure 2. Catalog core, key columns only (all columns are in the Database Schema document). A layer is unique per dataset, theme and LOD; a tileset is unique per layer and version.

Parent	Child	Foreign key	Cardinality
dataset	layer	dataset_id	1 to many (required)
layer	tileset	layer_id	1 to many (required)

Rules shown by this model: unique (dataset_id , theme_code , lod) on layer; unique (layer_id , version) on tileset; at most one is_active tileset per layer.

Import traceability

Every file placed into the system is an `import_job`. One file can fill several layers, so jobs and layers are linked through `import_job_layer`.

Figure 3. Import jobs and the layers they fill. Entities with only an id are shown for context.

Parent	Child	Foreign key	Cardinality
<code>dataset</code>	<code>import_job</code>	<code>dataset_id</code>	1 to many (required)
<code>import_job</code>	<code>import_job_layer</code>	<code>job_id</code>	1 to many (required)
<code>layer</code>	<code>import_job_layer</code>	<code>layer_id</code>	1 to many (required)
<code>import_job</code>	<code>layer</code>	<code>validated_by_job_id</code>	0..1 to many (optional)

A layer can move from draft to validated only when `validated_by_job_id` points to a succeeded job with passed validation that is linked to the layer through `import_job_layer`.

Link to the city model

The catalog never changes city objects. A layer is tied to its 3D objects by the **lineage tag** `dataset.theme.lodN`, written into `citydb.feature.lineage` at import time.

Figure 4. The dotted line is the lineage tag, not a foreign key. LOD is stored in `property.val_lod`.

Why a tag and not a link table

Option	Decision	Reason
Link table <code>layer_feature</code>	Rejected	Millions of rows that duplicate what 3DCityDB already stores
One schema per dataset	Rejected	Multiplies API, admin and statistics work
Tag in <code>feature.lineage</code>	Chosen	Native column, indexed by <code>feature_lineage_inx</code> , no extra rows

3DCityDB v5 facts the model relies on

Fact	Detail
Tables	17: <code>feature</code> , <code>property</code> , <code>geometry_data</code> , <code>implicit_geometry</code> , <code>appearance</code> , <code>surface_data</code> , <code>surface_data_mapping</code> , <code>appear_to_surface_data</code> , <code>tex_image</code> , <code>address</code> , <code>objectclass</code> , <code>datatype</code> , <code>namespace</code> , <code>codelist</code> , <code>codelist_entry</code> , <code>ade</code> , <code>database_srs</code>
LOD	Stored in <code>property.val_lod</code> (text) on geometry properties; <code>geometry_data</code> has no LOD column
LOD4 of CityGML 2.0	Kept as deprecated-namespace properties; rooms are class <code>BuildingRoom</code> (id 904)
Lineage	<code>feature.lineage</code> exists and is not indexed by default; the catalog adds <code>feature_lineage_inx</code>
Class ids	Building 901, BuildingPart 902, BuildingRoom 904, Road 607, Bridge 1001, TINRelief 502

Fact	Detail
SRID	Fixed per database at creation (<code>citydb.database_srs</code>)

Audit

Every insert, update and delete on catalog tables is logged with the signed-in user, the old row and the new row. The log is filled by triggers and cannot be edited.

Column	Meaning
at	When the change happened
app_user	Signed-in user who made it (mandatory)
table_name , pk	Which record changed
op	INSERT, UPDATE or DELETE
old_row , new_row	Full row before and after, as JSON

The audit log has no foreign keys. A record's history is found by `table_name` and `pk` .

Code lists

Code lists are small reference tables used as foreign keys instead of PostgreSQL ENUM types, so the admin UI can show drop-down lists and administrators can add values without a migration.

Code list	Values	Used by
<code>catalog.ref_lod_level</code>	0 to 4	<code>layer.lod</code>
<code>catalog.ref_theme</code>	building, bridge, road, terrain, vegetation, water, furniture, generic	<code>layer.theme_code</code>
<code>catalog.ref_generated_by</code>	external, naraga, manual	<code>dataset.generated_by</code>
<code>catalog.ref_layer_status</code>	draft, validated, published, archived	<code>layer.status</code>
<code>catalog.ref_tileset_status</code>	building, ready, failed, retired	<code>tileset.status</code>

Read model

View	Built from	Description
<code>catalog.v_layer</code>	layer, dataset, tileset, layer_feature_stats, citydb.feature	One row per layer with its active tileset, object count and stale flag. Used by the API and the inventory page.
<code>catalog.layer_feature_stats</code>	citydb.feature, objectclass, property	Materialized view: object counts per tag, class and LOD. Refreshed after each import.

Key indexes: `layer(loid, status)`, GiST on `layer.bbox` and `dataset.bbox`, `import_job(dataset_id, started_at desc)`, `change_log(table_name, pk, at desc)`, `change_log(app_user, at desc)`, `citydb.feature(lineage text_pattern_ops)`.